

UNIVERSIDAD TECNOLÓGICA NACIONAL
Tecnatura Universitaria en Programación a Distancia
Programación 2

Trabajo Práctico Integrador
Food Store – Sistema de Gestión de Pedidos de Comida

Estudiante: Ian Alexander Gunn

Legajo: 102623

Fecha de entrega: 20/06/2026

Índice

Índice	2
1. Marco Teórico	3
1.1. Tecnologías utilizadas	3
1.2. Conceptos clave aplicados	3
2. Decisiones Técnicas y Arquitectura	3
2.1. Organización en capas	4
2.2. Decisiones de diseño destacadas	4
2.3. Capturas de pantalla del sistema	4
3. Dificultades y Soluciones	8
4. Bibliografía y Webgrafía	8
5. Enlaces del Proyecto	8

1. Marco Teórico

El presente trabajo consiste en el desarrollo de Food Store, un sistema de consola para la gestión de pedidos de un negocio de comidas, implementado íntegramente en Java aplicando el paradigma de Programación Orientada a Objetos (POO).

1.1. Tecnologías utilizadas

Java 21: lenguaje de programación orientado a objetos sobre el que se desarrolló la totalidad del sistema.

Apache NetBeans: entorno de desarrollo integrado (IDE) empleado para escribir, compilar y ejecutar el proyecto, organizado como proyecto Maven.

Colecciones: el sistema no utiliza base de datos. Toda la información se mantiene en memoria durante la ejecución mediante listas dinámicas (ArrayList), que permiten almacenar y gestionar los objetos del dominio.

Nota sobre bases de datos relacionales: si bien el enunciado menciona el uso de JDBC en su título, esta versión del trabajo almacena los datos en memoria mediante colecciones, tal como se indica en el alcance del proyecto. Una evolución futura del sistema podría persistir los datos en una base de datos relacional (por ejemplo MySQL) utilizando JDBC para conectar la aplicación con el motor, ejecutar consultas SQL y mantener la información de forma permanente.

1.2. Conceptos clave aplicados

- Encapsulamiento: todos los atributos de las clases son privados y se acceden mediante getters y setters.
- Herencia: la clase abstracta Base concentra los atributos comunes (id, eliminado, createdAt) y todas las entidades del dominio heredan de ella.
- Interfaces: la interfaz Calculable define el comportamiento de cálculo de totales, implementado por la clase Pedido.
- Polimorfismo y sobrescritura: cada entidad sobrescribe el método toString() para producir listados legibles en consola.
- Composición: un Pedido crea y contiene sus propios objetos DetallePedido.
- Colecciones: se utilizan listas dinámicas para gestionar los conjuntos de entidades.
- Manejo de excepciones: se crean y capturan excepciones propias para validar reglas de negocio y evitar cierres inesperados del programa.

2. Decisiones Técnicas y Arquitectura

2.1. Organización en capas

El proyecto se organizó separando responsabilidades en tres capas, distribuidas en paquetes:

- Entidades (entities): el modelo de dominio basado en el UML provisto. Incluye la clase abstracta Base, las clases concretas Categoria, Producto, Usuario, DetallePedido y Pedido, y la interfaz Calculable.
- Servicios / lógica (service): contiene un servicio por entidad (CategoriaService, ProductoService, UsuarioService, PedidoService). Cada servicio mantiene su propia colección en memoria, genera los identificadores, ejecuta las operaciones CRUD y aplica las validaciones de negocio.
- Vista / consola (main): la clase Main contiene el menú principal y los submenús. Solo se ocupa de la interacción con el usuario (lectura por Scanner y mensajes), delegando toda la lógica en los servicios.

Adicionalmente, los enumerados (Rol, Estado, FormaPago) se ubican en el paquete enums, y las excepciones propias en el paquete exception.

2.2. Decisiones de diseño destacadas

- Generación de identificadores: cada servicio mantiene un contador de tipo Long que se incrementa con cada alta, garantizando ids únicos aun cuando se eliminen registros.
- Baja lógica (soft delete): al eliminar un registro no se quita de la colección, sino que se marca su atributo eliminado en true. Los listados y las búsquedas ignoran los registros eliminados, preservando el historial.
- Valores calculados: el subtotal de cada DetallePedido y el total de cada Pedido no se reciben por parámetro, sino que se calculan internamente. El total se obtiene mediante el método calcularTotal() de la interfaz Calculable.
- Validaciones en el servicio: las reglas de negocio (precio y stock no negativos, mail único, pedido con usuario, cantidad mayor a cero) se validan en la capa de servicios, que lanza la excepción correspondiente. El menú captura esas excepciones y muestra el mensaje al usuario.
- Creación transaccional del pedido: al crear un pedido se cargan primero todos sus detalles y, únicamente si el proceso se completa sin errores, el pedido se agrega a la colección. De este modo no quedan datos inconsistentes en la memoria.

2.3. Capturas de pantalla del sistema

Menú principal:

```
=== SISTEMA DE PEDIDOS (FOOD STORE) ===  
1. Categorías  
2. Productos  
3. Usuarios  
4. Pedidos  
0. Salir  
Seleccione:
```

Crear categoría:

```
=== SISTEMA DE PEDIDOS (FOOD STORE) ===  
1. Categorías  
2. Productos  
3. Usuarios  
4. Pedidos  
0. Salir  
Seleccione: 1  
  
--- CATEGORÍAS ---  
1. Listar  
2. Crear  
3. Editar  
4. Eliminar  
0. Volver  
Seleccione: 2  
Nombre: Pizzas  
Descripción: Variedad de pizzas a la piedra  
Categoría creada con éxito. ID generado: 1
```

Listar categorías:

```
--- CATEGORÍAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1
ID 1 | Pizzas | Variedad de pizzas a la piedra
ID 2 | Bebidas | Gaseosas, aguas y jugos
ID 3 | Hamburguesas | Hamburguesas artesanales
ID 4 | Postres | Helados y tortas
```

Listar productos:

```
--- PRODUCTOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1
ID 1 | Muzzarella | $8500.0 | Stock: 20 | Categoría: Pizzas
ID 2 | Napolitana | $9200.0 | Stock: 15 | Categoría: Pizzas
ID 3 | Coca-Cola 1.5L | $2500.0 | Stock: 40 | Categoría: Bebidas
ID 4 | Cheeseburger | $7800.0 | Stock: 25 | Categoría: Hamburguesas
```

Crear usuario:

```
=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione: 3

--- USUARIOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 2
Nombre: Juan
Apellido: Pérez
Mail: juanperez@gmail.com
Celular: 1145678900
Contraseña: 123456
Rol: 1. ADMIN 2. USUARIO
Seleccione: 2
Usuario creado con éxito. ID generado: 1
```

Crear pedido:

```
=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione: 4

--- PEDIDOS ---
1. Listar
2. Crear
3. Actualizar estado / forma de pago
4. Eliminar
0. Volver
Seleccione: 2
Usuarios disponibles:
ID 1 | Juan Pérez | juanperez@gmail.com | Rol: USUARIO
ID del usuario: 1
Forma de pago: 1. TARJETA 2. TRANSFERENCIA 3. EFECTIVO
Seleccione: 1

Productos disponibles:
ID 1 | Muzzarella | $8500.0 | Stock: 20 | Categoría: Pizzas
ID 2 | Napolitana | $9200.0 | Stock: 15 | Categoría: Pizzas
ID 3 | Coca-Cola 1.5L | $2500.0 | Stock: 40 | Categoría: Bebidas
ID 4 | Cheeseburger | $7800.0 | Stock: 25 | Categoría: Hamburguesas
ID del producto a agregar: 1
Cantidad: 2
Producto agregado al pedido.
¿Agregar otro producto? (S/N): s

Productos disponibles:
ID 1 | Muzzarella | $8500.0 | Stock: 20 | Categoría: Pizzas
ID 2 | Napolitana | $9200.0 | Stock: 15 | Categoría: Pizzas
ID 3 | Coca-Cola 1.5L | $2500.0 | Stock: 40 | Categoría: Bebidas
ID 4 | Cheeseburger | $7800.0 | Stock: 25 | Categoría: Hamburguesas
ID del producto a agregar: 3
Cantidad: 3
Producto agregado al pedido.
¿Agregar otro producto? (S/N): n
Pedido creado con éxito. ID: 1 | Total: $24500.0
```

Mensaje de error de validación:

```
--- PRODUCTOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 2
Categorías disponibles:
ID 1 | Pizzas | Variedad de pizzas a la piedra
ID 2 | Bebidas | Gaseosas, aguas y jugos
ID 3 | Hamburguesas | Hamburguesas artesanales
ID 4 | Postres | Helados y tortas
ID de la categoría: 4
Nombre: Helado de chocolate
Descripción: Chocolate al 70%
Precio: -10000
Stock: 30
Imagen (URL o nombre de archivo): heladochoco.jpg
¿Disponible? (S/N): s
Error: El precio no puede ser negativo.
```

3. Dificultades y Soluciones

Durante el desarrollo del trabajo surgieron principalmente dos dificultades técnicas, que se resolvieron a medida que se avanzaba.

Aplicación de las reglas de negocio: una de las partes más desafiantes fue trasladar las reglas de negocio del enunciado al código de la capa de servicios. No alcanzaba con realizar el CRUD básico, sino que cada operación debía respetar condiciones específicas: que el precio y el stock de un producto no fueran negativos, que el mail de cada usuario fuera único, que no se pudiera crear un pedido sin usuario y que la cantidad de cada detalle fuera mayor a cero. La dificultad estaba en decidir dónde ubicar cada validación y cómo informar el error sin cortar la ejecución del programa. Se resolvió concentrando todas las validaciones dentro de los servicios, que lanzan una excepción propia cuando una regla no se cumple, mientras que el menú captura esa excepción con un bloque try/catch y muestra un mensaje al usuario. De esta forma se respetó la separación de responsabilidades entre la lógica y la vista.

Olvido de la herencia en las clases concretas: al crear las entidades del modelo, en varias ocasiones se omitió la cláusula extends Base en la declaración de las clases. Esto provocaba errores de compilación, ya que las clases quedaban sin los atributos heredados (id, eliminado, createdAt). El problema se soluciona incorporando la herencia y la llamada al constructor de la clase padre mediante super(id) en la primera línea del constructor de cada subclase. A partir de esta experiencia se adoptó como criterio verificar siempre, al crear una nueva entidad, que extendiera correctamente de la clase Base.

4. Bibliografía y Webgrafía

- Material teórico y apuntes de la cátedra de Programación 2 (UTN), correspondientes a las unidades de Programación Orientada a Objetos, Constructores y Relaciones entre clases, UML, Colecciones y Enumeraciones, Herencia y Polimorfismo, e Interfaces y Excepciones.

5. Enlaces del Proyecto

Repositorio GitHub: <https://github.com/ianagunn/food-store-tpi-programacion2>

Video demostrativo: <https://youtu.be/1OVEtdLcS9s>